

10 Snowflake Queries That Waste Thousands of Dollars

Snowflake makes it dangerously easy to scale compute.

That convenience is powerful — but it also means inefficient SQL can silently burn through thousands of dollars every month.

Many teams focus only on correctness: - “Does the query work?” - “Does the dashboard load?” - “Did the pipeline finish?”

But the real question should be:

“How much warehouse compute did this query consume?”

Here are 10 common Snowflake query patterns that quietly destroy performance and increase cloud costs — along with better alternatives.

1. SELECT * on Large Tables

The Problem

```
SELECT *  
FROM SALES_TRANSACTIONS;
```

This looks harmless.

But on wide enterprise tables with hundreds of columns and billions of rows, Snowflake scans unnecessary data, increasing: - I/O - Memory usage - Network transfer - Result cache size

Even though Snowflake uses columnar storage, selecting unused columns still increases scan costs.

Better Approach

```
SELECT  
    customer_id,  
    order_id,  
    total_amount  
FROM SALES_TRANSACTIONS;
```

Only fetch what you need.

Why It Saves Money

- Fewer micro-partitions scanned

- Lower memory consumption
 - Faster execution time
 - Smaller result sets
-

2. Missing WHERE Clauses on Huge Tables

The Problem

```
SELECT COUNT(*)  
FROM EVENT_LOGS;
```

If your table contains years of data, this query scans everything.

Teams often run queries like this repeatedly from dashboards, notebooks, or monitoring jobs.

Better Approach

```
SELECT COUNT(*)  
FROM EVENT_LOGS  
WHERE event_date >= CURRENT_DATE - 7;
```

Always reduce the scanned dataset.

Cost Impact

Scanning 3 years of logs instead of 7 days can multiply compute costs by hundreds of times.

3. Using DISTINCT Everywhere

The Problem

```
SELECT DISTINCT customer_id  
FROM ORDERS;
```

DISTINCT forces expensive sorting or hashing operations.

Many developers use it as a “quick fix” for duplicate rows instead of solving the root cause.

Better Approach

Fix duplication at the join or source level.

Example:

```
SELECT customer_id
FROM ORDERS
GROUP BY customer_id;
```

Or better: - Deduplicate upstream - Use proper join conditions - Enforce business keys

Why It's Expensive

DISTINCT often causes: - Large memory spills - Remote disk usage - Longer warehouse runtime

4. Exploding CROSS JOINs

The Problem

```
SELECT *
FROM customers c
CROSS JOIN products p;
```

A CROSS JOIN multiplies every row from one table with every row from another.

10 million customers $\times$ 100,000 products = disaster.

Better Approach

Use explicit join conditions:

```
SELECT *
FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id;
```

Cost Impact

Accidental Cartesian products are one of the fastest ways to: - Spike warehouse usage - Cause memory spills - Crash dashboards

5. Repeated CTE Recalculation

The Problem

```
WITH expensive_data AS (  
    SELECT *  
    FROM huge_table  
    WHERE event_date >= CURRENT_DATE - 365  
)  
  
SELECT COUNT(*) FROM expensive_data;  
  
SELECT AVG(amount) FROM expensive_data;
```

Depending on optimization behavior, Snowflake may recompute expensive CTEs multiple times.

Better Approach

Materialize intermediate results:

```
CREATE TEMP TABLE expensive_data AS  
SELECT *  
FROM huge_table  
WHERE event_date >= CURRENT_DATE - 365;
```

Then reuse the temp table.

Why It Saves Money

- Avoids repeated scans
 - Reduces redundant compute
 - Improves pipeline stability
-

6. Inefficient Window Functions

The Problem

```
SELECT
    *,
    ROW_NUMBER() OVER (
        PARTITION BY customer_id
        ORDER BY transaction_time
    ) AS rn
FROM transactions;
```

Window functions are expensive on massive datasets.

Especially when: - Partition keys are high cardinality - Ordering columns are huge - No filtering is applied beforehand

Better Approach

Filter early:

```
WITH recent_transactions AS (
    SELECT *
    FROM transactions
    WHERE transaction_date >= CURRENT_DATE - 30
)

SELECT
    *,
    ROW_NUMBER() OVER (
        PARTITION BY customer_id
        ORDER BY transaction_time
    ) AS rn
FROM recent_transactions;
```

Cost Impact

Window functions can trigger: - Massive sorting operations - Local disk spills - Remote storage spills

7. ORDER BY Without LIMIT

The Problem

```
SELECT *  
FROM large_table  
ORDER BY created_at DESC;
```

Sorting billions of rows just to inspect recent data wastes compute.

Better Approach

```
SELECT *  
FROM large_table  
ORDER BY created_at DESC  
LIMIT 100;
```

Why It Saves Money

Sorting is one of the most expensive operations in distributed systems.

Adding LIMIT dramatically reduces workload.

8. Querying Semi-Structured Data Repeatedly

The Problem

```
SELECT  
    payload:user:id::STRING,  
    payload:device:type::STRING  
FROM raw_events;
```

Repeated JSON parsing is expensive at scale.

Better Approach

Flatten and normalize frequently used fields into structured columns.

Example:

```
CREATE TABLE curated_events AS
SELECT
    payload:user:id::STRING AS user_id,
    payload:device:type::STRING AS device_type
FROM raw_events;
```

Why It Saves Money

Structured columns: - Compress better - Scan faster - Avoid repeated parsing overhead

9. Running Tiny Queries on Huge Warehouses

The Problem

Using an XL warehouse for lightweight queries:

```
SELECT CURRENT_TIMESTAMP;
```

Warehouse size matters.

Even trivial queries consume credits based on warehouse size.

Better Approach

Use warehouse sizing strategically: - XS for BI - Medium for ELT - Large only for heavy transformations

Enable: - Auto suspend - Auto resume

Real-World Issue

Many organizations lose thousands monthly because warehouses stay running overnight.

10. Ignoring Query Profile and Spills

The Problem

Teams rarely inspect Snowflake Query Profile.

As a result, they miss: - Remote disk spills - Large scans - Bad joins - Inefficient repartitioning

Better Approach

Regularly inspect: - Bytes scanned - Partitions scanned - Spill volume - Execution time - Join operations

Use:

```
SELECT *  
FROM TABLE(INFORMATION_SCHEMA.QUERY_HISTORY())  
ORDER BY start_time DESC;
```

Why It Saves Money

The Query Profile often reveals: - Queries scanning terabytes unnecessarily - Warehouses oversized for workloads - Inefficient transformations

Bonus Tips That Save Serious Money

Enable Auto Suspend

Recommended: - 60 seconds - 300 seconds max

Use Clustering Carefully

Bad clustering keys can increase maintenance costs.

Cluster only: - Very large tables - Frequently filtered tables

Use Result Cache

Snowflake automatically caches results.

Avoid changing queries unnecessarily if dashboards repeat the same logic.

Final Thoughts

In Snowflake, performance problems are usually cost problems.

The most expensive query is often not: - the slowest one, - the most complex one, - or the biggest transformation.

It's the query that runs inefficiently thousands of times every day.

Small SQL optimizations can save: - hours of runtime, - warehouse contention, - and thousands of dollars per month.

The best Snowflake engineers don't just write correct SQL.

They write cost-efficient SQL.

Suggested Tags for Medium

- Snowflake
- SQL
- Data Engineering
- Cloud Computing
- Big Data
- Analytics Engineering